

Playwright Automation Framework – Week 5 Report

1. Project Title

UI Automation Testing Framework using Playwright – Week 5

2. Objective

The objective of this project is to design and implement a scalable UI automation framework using Playwright and TypeScript to validate flakiness, adding logs, attaching evidences, artifacts and checking return & refund implementation.

Target Website:

[SDET-retail-demo-app](#)

3. User Story

User Story 1 – Flakiness

Making a test failed intentionally for flakiness and correcting the mistake.

User Story 2 – Making network offline

Turn the network to offline and hit the sale, it goes to pending state.

When connected back to internet, the sale automatically synced.

User Story 3 – Adding logs to the checkout flow

Adding a product to the cart, moving to checkout and placing the order.

User Story 4 – Checking returns and refunds in API

Checking returns and partial refunds in multi order.

Validating the refund amount and tax amount refunded.

Checking with same idempotency key

4. Environment & Setup

Hardware

- **System:** Windows
- **RAM:** Minimum 8 GB
- **Processor:** Intel i5 or above

Software

Component Version

Node.js **Latest LTS**

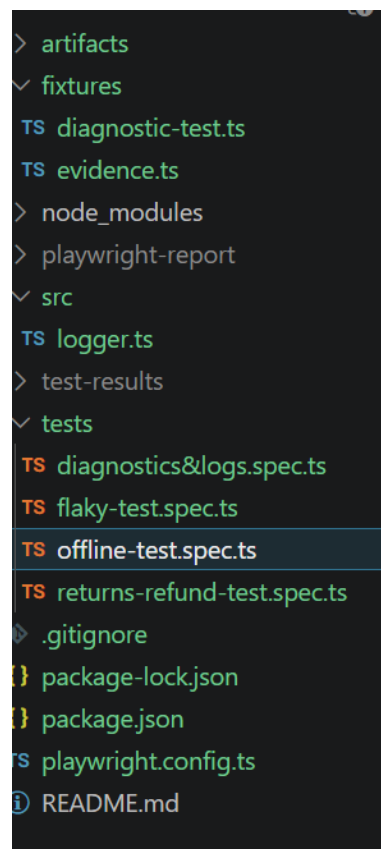
Playwright **Latest**

TypeScript **Latest**

VS Code **Latest**

Browser **Chromium**

5. Project Structure



```
> artifacts
✓ fixtures
TS diagnostic-test.ts
TS evidence.ts
> node_modules
> playwright-report
✓ src
TS logger.ts
> test-results
✓ tests
TS diagnostics&logs.spec.ts
TS flaky-test.spec.ts
TS offline-test.spec.ts
TS returns-refund-test.spec.ts
> .gitignore
} package-lock.json
} package.json
TS playwright.config.ts
i README.md
```

6. Scenarios Covered

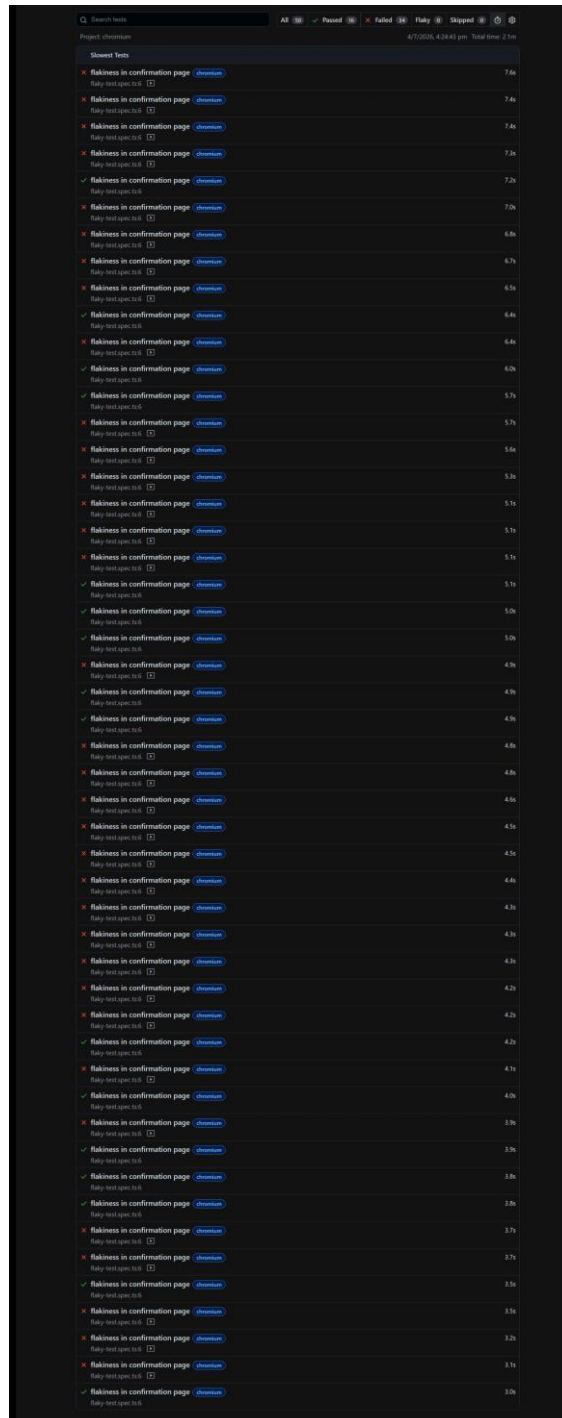
Scenario 1 – Flakiness

Expected:

- Some times test passes, sometime fails when run repeatedly

Status:

Sometimes pass, sometimes fail



The screenshot displays a test runner interface with a dark theme. At the top, there is a search bar and a filter bar showing 'All' tests, with counts for 'Passed', 'Failed', 'Flaky', and 'Skipped'. Below this, a table lists individual test runs. Each row includes a status icon (a green checkmark for passed or a red 'X' for failed), the test name 'flakiness in confirmation page', a link to view details, and a duration. The tests are grouped under the heading 'Slowest tests'. The results show a mix of passed and failed tests, illustrating the flakiness of the scenario.

Search tests	All	Passed	Failed	Flaky	Skipped	4/7/2020, 4:24:03 pm Total time: 2 hrs
Slowest tests						
✗	flakiness in confirmation page	chromium				7.6s
	flakiness in confirmation page	chromium				7.6s
✗	flakiness in confirmation page	chromium				7.6s
✗	flakiness in confirmation page	chromium				7.3s
✓	flakiness in confirmation page	chromium				7.2s
✗	flakiness in confirmation page	chromium				7.0s
✗	flakiness in confirmation page	chromium				6.9s
✗	flakiness in confirmation page	chromium				6.7s
✗	flakiness in confirmation page	chromium				6.5s
✓	flakiness in confirmation page	chromium				6.4s
✗	flakiness in confirmation page	chromium				6.4s
✓	flakiness in confirmation page	chromium				6.0s
✓	flakiness in confirmation page	chromium				5.7s
✗	flakiness in confirmation page	chromium				5.7s
✗	flakiness in confirmation page	chromium				5.6s
✗	flakiness in confirmation page	chromium				5.3s
✗	flakiness in confirmation page	chromium				5.1s
✗	flakiness in confirmation page	chromium				5.1s
✗	flakiness in confirmation page	chromium				5.1s
✓	flakiness in confirmation page	chromium				5.1s
✓	flakiness in confirmation page	chromium				5.0s
✓	flakiness in confirmation page	chromium				5.0s
✗	flakiness in confirmation page	chromium				4.9s
✓	flakiness in confirmation page	chromium				4.9s
✓	flakiness in confirmation page	chromium				4.9s
✗	flakiness in confirmation page	chromium				4.8s
✗	flakiness in confirmation page	chromium				4.8s
✗	flakiness in confirmation page	chromium				4.8s
✗	flakiness in confirmation page	chromium				4.5s
✗	flakiness in confirmation page	chromium				4.5s
✗	flakiness in confirmation page	chromium				4.4s
✗	flakiness in confirmation page	chromium				4.3s
✗	flakiness in confirmation page	chromium				4.2s
✗	flakiness in confirmation page	chromium				4.2s
✓	flakiness in confirmation page	chromium				4.2s
✗	flakiness in confirmation page	chromium				4.1s
✓	flakiness in confirmation page	chromium				4.0s
✗	flakiness in confirmation page	chromium				3.9s
✓	flakiness in confirmation page	chromium				3.8s
✓	flakiness in confirmation page	chromium				3.8s
✗	flakiness in confirmation page	chromium				3.7s
✗	flakiness in confirmation page	chromium				3.7s
✓	flakiness in confirmation page	chromium				3.5s
✗	flakiness in confirmation page	chromium				3.5s
✗	flakiness in confirmation page	chromium				3.2s
✗	flakiness in confirmation page	chromium				3.1s
✓	flakiness in confirmation page	chromium				3.0s

Evidence:

Correction: Adding await when clicking and “Place order” button and removing timeout.

Scenario 2 – Making network offline and checking sales

Expected:

- Making network offline and hitting sale, it goes into pending state and when connected back sync automatically

Status:

PASS

Evidence:

The screenshot displays the Cypress test runner interface. At the top, a search bar is followed by a summary bar showing 'All 1', 'Passed 1', 'Failed 0', 'Flaky 0', and 'Skipped 0'. The test title 'offline test and then making online and checking sync' is prominently displayed, along with the file path 'offline-test.spec.ts:5' and a duration of '1.6s'. Below the title, a 'chromium' browser icon and a 'Run' button are visible. The 'Test Steps' section is expanded, showing a list of 16 steps, each with a green checkmark indicating success and a duration. The steps include 'Before Hooks', 'Navigate to "/pos"', 'Reload', 'Set offline mode', several 'Expect' assertions for visibility, text, and state, a 'Click' action on a 'Queue sale' button, and 'After Hooks'. Below the test steps, the 'Attachments' section shows a single attachment named 'stdout'. At the bottom, it indicates the test was 'Executed in Worker #0'.

Step	Duration
Before Hooks	697ms
Navigate to "/pos" — offline-test.spec.ts:7	410ms
Reload — offline-test.spec.ts:8	57ms
Set offline mode — offline-test.spec.ts:10	9ms
Expect "toBeVisible" getByText('Offline - sales will be queued') — offline-test.spec.ts:11	29ms
Expect "toBeFalsy" — offline-test.spec.ts:12	0ms
Evaluate — offline-test.spec.ts:13	5ms
Expect "toBe" — offline-test.spec.ts:14	0ms
Click getByRole('button', { name: 'Queue sale' }) — offline-test.spec.ts:16	37ms
Expect "toHaveText" getByTestId('outbox-count') — offline-test.spec.ts:18	5ms
Set offline mode — offline-test.spec.ts:20	1ms
Expect "toBeVisible" getByText('Online - sales sync normally') — offline-test.spec.ts:22	14ms
Expect "toHaveText" getByTestId('outbox-count') — offline-test.spec.ts:23	8ms
After Hooks	637ms

Scenario 3 – Adding logs to the checkout flow

Expected:

- Adding logs for each step for the entire checkout flow.

Status:

PASS

Q Filter steps		
> ✓ Before Hooks		827ms
> ✓ Navigate to "/catalog" — diagnostics-logs.spec.ts:5		436ms
> ✓ Click getByRole('link', { name: 'View Running Shoes' }) — diagnostics-logs.spec.ts:6		199ms
> ✓ Click getByRole('button', { name: 'Add to cart' }) — diagnostics-logs.spec.ts:8		88ms
> ✓ Click getByRole('button', { name: 'Proceed to checkout' }) — diagnostics-logs.spec.ts:10		92ms
> ✓ Fill "UST10" locator("#coupon-code") — diagnostics-logs.spec.ts:12		34ms
> ✓ Expect "toContainText" locator('order-summary') — diagnostics-logs.spec.ts:13		12ms
> ✓ Click getByRole('button', { name: 'Place order' }) — diagnostics-logs.spec.ts:15		57ms
> ✓ Expect "toContainText" locator("#confirmation-title") — diagnostics-logs.spec.ts:16		7ms
> ✓ After Hooks		661ms

Evidence:

```
▼ stdout
[INFO] 2026-07-04T11:35:22.130Z - test started
[INFO] 2026-07-04T11:35:22.132Z - Checkout journey
[INFO] 2026-07-04T11:35:22.773Z - Opened Product Detail Page
[INFO] 2026-07-04T11:35:22.862Z - cart page opened
[INFO] 2026-07-04T11:35:22.956Z - checkout page opened
[INFO] 2026-07-04T11:35:23.015Z - coupon applied
[INFO] 2026-07-04T11:35:23.082Z - order placed
[INFO] 2026-07-04T11:35:23.085Z - test finished {"status":"passed"}
```

Scenario 4 – Checking returns and refunds in API

Expected:

Checking returns and partial refunds in multi order.

Validating the refund amount and tax amount refunded.

Checking for over refund when quantity is more than the ordered quantity

Checking with same idempotency key

Status:

PASS

Evidence:

Q Search tests

All 1Passed 1Failed 0Flaky 0Skipped 0

A partial refund testing

returns&refund-test.spec.ts:31

chromium

Run

1.7s

Test Steps

Filter steps

> ✓ Before Hooks666ms

> ✓ POST "/api/refund-lab/orders" — returns&refund-test.spec.ts:833ms

> ✓ Expect "toBeTruthy" — returns&refund-test.spec.ts:240ms

> ✓ Attach "order created" — ./fixtures/evidence.ts:112ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:380ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:390ms

> ✓ POST "/api/refunds/check" — returns&refund-test.spec.ts:443ms

> ✓ Attach "refund approval" — ./fixtures/evidence.ts:111ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:610ms

> ✓ POST "/api/refunds" — returns&refund-test.spec.ts:693ms

> ✓ Attach "refund status" — ./fixtures/evidence.ts:111ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:870ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:880ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:890ms

> ✓ GET "/api/refund-lab/orders/7043" — returns&refund-test.spec.ts:944ms

> ✓ Attach "ledger" — ./fixtures/evidence.ts:111ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:1031ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:1040ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:1050ms

> ✓ POST "/api/refunds" — returns&refund-test.spec.ts:1103ms

> ✓ Attach "No over-refund" — ./fixtures/evidence.ts:111ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:1280ms

> ✓ POST "/api/refunds" — returns&refund-test.spec.ts:1332ms

> ✓ Attach "same idempotency key" — ./fixtures/evidence.ts:111ms

> ✓ Expect "toBe" — returns&refund-test.spec.ts:1510ms

> ✓ Navigate to "/returns?orderId=7043" — returns&refund-test.spec.ts:155447ms

> ✓ Expect "toHaveText" getByTestId("refund-total") — returns&refund-test.spec.ts:15773ms

> ✓ Screenshot — returns&refund-test.spec.ts:159133ms

> ✓ After Hooks541ms

Attachments

> order created

> refund approval

> refund status

> ledger

> No over-refund

> same idempotency key

> stdout

> Executed in Worker #0

7. Execution Steps

Install dependencies:

npm install

Install browsers:

npx playwright install

Execute tests:

npx playwright test

Open report:

npx playwright show-report

8. Test Results

Execution Summary:

Total Tests: 4

Passed: 4

Failed: 0

Execution Mode:

Chromium Browser

Execution Duration:

~8 seconds

9. Evidence

Execution Output:

Running 4 tests using 4 workers

✓ Flakiness

✓ Network offline

✓ logs

✓ returns and refunds

4 passed

HTML Report Generated Successfully

Screenshots:

Stored under:

test-results/

Video recordings on failure

Trace:

Enabled on first retry

10. Environment Configuration

Playwright Configuration:

- **Browser: Chromium**
- **Trace Collection: Enabled**
- **Base URL Configured**
- **HTML Reporting Enabled**

Configuration Parameters:

- **fullyParallel**
- **retries**
- **workers**
- **trace**
- **baseURL**

11. Conclusion

The Playwright automation framework was successfully implemented using Fixtures and tests.